

CSCI 532 Project Paper

Tyriz Newton
Rochester Institute of Technology
tn1207@rit.edu

Kai Fan
Rochester Institute of Technology
kf5601@rit.edu

Abstract

Machine learning used to use classical data learning that relied on having one central server analyzing all of the data provided, which led to many privacy and efficiency concerns. This led to the creation of federated learning, which uses trained data from mobile devices or IoTs to allow for data to not reveal participants' private information. There are many different algorithms and aggregators that are used to increase the computational power of federated learning, like FedAvg or FedProx.

1. Introduction

Machine learning models' main method of training used to be centralized data learning, which had all of the data be trained at the central server. This form of training was susceptible to privacy threats on participants' data because of its centralization of data on a single server and deep learning models' habit of storing intricate information, leading to the models storing details about participants' data for privacy attacks to easily access [1]. This led to the development of federated learning by Google in 2017, which trained data that was distributed on mobile devices or IoTs and "learned a shared model by aggregating locally-computed updates" [2]. With federated learning, participants' data is much less susceptible to privacy attacks while also increasing bandwidth capacity.

1.1 Classical Vs. Federated

In their paper comparing federated learning to classical and distributed machine learning, the Department of Computer Science at Nagoya Institute of Technology found federated learning to outperform the other two learning models by more than 15% in terms of accuracy when analyzing datasets [3]. This was due to centralized machine learning requiring the data to be repeatedly uploaded back to the central server while the distributed machine learning struggled with security issues. Federated learning is also preferable to centralized learning when there are skewed data distributions, complex models, and high data dimensionality [4].

As stated before, federated learning was seen as a general upgrade to classical machine learning, and the reason for that is its protection of user data. However, federated learning is better than classical learning specifically because of its new features and the algorithms it uses that help it to be more accurate and private than classical learning. The three main privacy features of federated learning are model aggregation, homomorphic encryption, and differential privacy [5]. Model aggregation trains the central server's model by summarizing

the model's parameters from all parties, which helps to avoid using the original data. Homomorphic encryption protects the data storage by making it possible for users to process the encrypted data while not disclosing any original data. This works with federated learning by making it possible for the model to be encrypted while the central server transmits data to the other IoTs. Lastly, differential privacy makes the calculation results of the training insensitive to the changes of the data's records, making it difficult for attackers to obtain information by observing the results. In federated learning, adding "noise" can help offset the data's vulnerabilities.

1.2 Aggregators & Algorithms

Federated learning has key aggregation methods and algorithms that are used to help speed up the process of machine learning. The most widely used aggregation method is federated averaging, or FedAvg. Federated averaging works by having each client train locally while the central server calculates a weighted average of client model weight. This method was patented by Google in 2017 under the patent order EP3660754A1, and it was created with the purpose of reducing communication costs and handling unbalanced data [6]. FedAvg's efficiency made it the main method of aggregating data in federated learning; however, it also has its drawbacks. The method struggles with its convergence rate when using non-IID data, and the theoretical output of FedAvg is lowered when considering the disconnection of IoTs during training [7]. There have been studies that try to improve on FedAvg, like the study done on FedCostWAvG that includes the amount of the cost function's decrease in FedAvg's formula, increasing the performance of the aggregator [8].

Another aggregator that aims to improve on FedAvg is FedProx, which is very similar to FedAvg but differs in its addition of a proximal regularization term that allows for both statistical and systems heterogeneity. This proximal term creates local solvers for each IoT, which then sends signals to the central server after the data is calculated. FedProx has shown to have a 22% increase in absolute test accuracy vs

FedAvg's performance, proving its improvement on the aggregator [9].

Stochastic Controlled Averaging for Federated Learning, or SCAFFOLD, is another aggregator that focuses on fixing the issue of local updates diverging from the central server by using control variates. These control variates are correction

terms that both the IoTs and central server use to adjust local gradient updates, which help with pulling the local training back towards the central server. This makes SCAFFOLD useful in analyzing similarities between client data; however, it falls short in its doubling of communication usage [10].

1.3 FL Feature Analysis vs. Centralized Learning

Table 1. Comparison of federated learning aggregation techniques and centralized learning.

Technique	Year / Source	Computational Efficiency	Robustness to Attacks	Scalability (large # clients)	Communication Overhead
Centralized Learning	1950s	High (single-pass training, no aggregation step)	None — single point of failure; no distributed poisoning risk but no data isolation either	Poor — bottlenecked by central server compute/storage as data volume grows	None (no client-server round trips)
FedAvg	McMahan et al., 2017 [2]	Very high — $O(n)$ weighted average, negligible server cost	None — a single malicious client can arbitrarily skew the global model	Excellent — designed for millions of clients	Low — the baseline FL communicates once per round
FedMedian (Coordinate-wise Median)	Yin et al., 2018 [11]	High — $O(n \log n)$ per coordinate (sort-based)	Moderate — resists some outliers but treats each parameter independently, missing correlated attacks	Good — cheap enough to scale, though sorting cost grows with client count	Low — same payload as FedAvg
Trimmed Mean	Yin et al., 2018 [11]	High — $O(n \log n)$, similar cost to Median	Moderate — degrades badly under “omniscient” attacks that know the trim threshold	Good — comparable cost profile to FedMedian	Low
Krum / Multi-Krum	Blanchard et al., 2017 [12]	Low — $O(n^2d)$, expensive pairwise-distance computation across all clients	High — formally proven Byzantine-resilient (selects the most “central” update)	Poor at scale — quadratic cost makes large client pools expensive	Low (selection happens server-side)
Bulyan	El Mhamdi et al., 2018 [13]	Very low — combines Krum's selection with trimmed-mean filtering, compounding cost	Very high — explicitly designed to fix weaknesses where Krum/RFA alone can still be steered to an ineffectual model	Poor — the most computationally expensive of the classic methods	Low
RFA (Robust Federated Averaging / Geometric Median)	Pillutla et al., 2019–2022 [14]	Moderate — iterative geometric-median computation	High — robust to corrupted/poisoned updates via median-based geometry	Moderate — scales better than Krum/Bulyan but worse than simple averaging	Higher — roughly 3x standard FedAvg communication cost
FedLAW	Li et al., 2023 [15]	Moderate — learns aggregation weights via a server-side proxy dataset rather than uniform averaging	Improves generalization under heterogeneity; not designed as a Byzantine defense	Good — comparable cost to FedAvg with an added proxy-dataset step	Moderate
Bayesian Robust Aggregation	Karakulev et al., 2025 [16]	Moderate — Bayesian inference step adds overhead vs. simple averaging, but avoids Krum's quadratic blowup	High — treats aggregation as posterior inference over “true” client update, benchmarked against FedAvg/Median/RFA/Trimmed Mean/Multi-Krum	Good	Moderate
TinyGuard	Mahdavi et al., 2026 [17]	High — explicitly designed as a lightweight defense for resource-constrained devices	Moderate–High — comparable robustness to Krum/Trimmed Mean/Bulyan at a fraction of the compute cost	Excellent — built specifically for scalability on constrained/IoT-style clients	Low

3. Challenges and Future Directions

As federated learning is a relatively new method of machine learning, there have been challenges in making sure it works in every situation. This has led to research being done in hopes of expanding federated learning into different

forms of learning using new fusion methods and other learning paradigms. Research has been done into figuring out if federated learning can be used with "multitask learning, meta-learning, transfer learning, unsupervised learning, and reinforcement learning" [18]. It's been found that transfer learning can be useful for reusing information, improving data utilization and reducing the need for repeated training.

Unsupervised learning has also shown some promise for federated learning, as it allows for the IoTs to calculate data patterns across datasets [18]. Meta-learning, otherwise known as FedMeta, is the use of a parameterized algorithm instead of a central server like move federated learning structures. This has shown success in reducing communication costs and increasing convergence rates [19]. As for an improvement in communication protocol, some users of federated learning have taken to using Gradient Compression technologies, as they can help reduce communication overhead while maintaining federated learning model training accuracy [20]. While this has shown its use in helping with communication, it struggles with efficient energy usage, as large, varied datasets are a common challenge for Gradient Compression and Federated Learning in general. As it stands, federated learning has many new ideas that benefit its ability to train data, however it struggles to find a concrete upgrade that improves every facet of its abilities without diminishing another.

4. References

- [1] B. Rao, J. Zhang, D. Wu, C. Zhu, X. Sun, and B. Chen, "Privacy inference attack and defense in centralized and federated learning: A comprehensive survey," *IEEE Trans. Artif. Intell.*, vol. 6, no. 2, pp. 333–353, Feb. 2025.
- [2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2017.
- [3] M. Asad, A. Moustafa, and T. Ito, "Federated learning versus classical machine learning: A convergence comparison," May 2021. doi: 10.22541/au.162074596.66890690/v1.
- [4] S. Garst, J. Dekker, and M. Reinders, "A comprehensive experimental comparison between federated and centralized learning," *Database*, vol. 2025, 2025. doi: 10.1093/database/baaf016.
- [5] C. Zhang et al., "A survey on federated learning," *Knowledge-Based Syst.*, vol. 216, p. 106775, Mar. 2021. doi: 10.1016/j.knosys.2021.106775.
- [6] H. B. McMahan, D. M. Bacon, J. Konečný, and X. Yu, "Communication efficient federated learning," *European Patent Application EP3660754A1*, Mar. 6, 2020.
- [7] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020. doi: 10.48550/arXiv.1907.02189.
- [8] L. Mächler et al., "FedCostWAVg: A new averaging for better federated learning," in *Lecture Notes in Computer Science*, 2022, pp. 383–391. doi: 10.1007/978-3-031-09002-8_34.
- [9] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems (MLSys)*, 2020. doi: 10.48550/arXiv.1812.06127.
- [10] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020.
- [11] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018.
- [12] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [13] E. M. El Mhamdi, R. Guerraoui, and S. Rouault, "The hidden vulnerability of distributed learning in Byzantium," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018.
- [14] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Trans. Signal Process.*, vol. 70, pp. 1142–1154, 2022.
- [15] Z. Li, T. Lin, X. Shang, and C. Wu, "Revisiting weighted aggregation in federated learning with neural networks," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR vol. 202, 2023, pp. 19767–19788.
- [16] A. Karakulev, U. Zafar, S. Toor, and P. Singh, "Bayesian robust aggregation for federated learning," *arXiv:2505.02490*, May 2025.
- [17] A. Mahdavi, S. Aghapour, A. Zamanifar, and A. Farhadi, "TinyGuard: A lightweight Byzantine defense for resource-constrained federated learning via statistical update fingerprints," *arXiv:2602.02615*, Feb. 2026.
- [18] S. Ji et al., "Emerging trends in federated learning: From model fusion to federated X learning," *Int. J. Machine Learning and Cybernetics*, vol. 15, no. 9, pp. 3769–3790, Apr. 2024. doi: 10.1007/s13042-024-02119-1.
- [19] F. Chen, M. Luo, Z. Dong, Z. Li, and X. He, "Federated meta-learning with fast convergence and efficient communication," Dec. 2019.
- [20] L. Xing et al., "A survey of federated learning-based gradient compression for internet of vehicles," *Eng. Appl. Artif. Intell.*, vol. 159, p. 111662, Nov. 2025. doi: 10.1016/j.engappai.2025.111662.

Section B: Data Preparation Strategy

The data-preparation strategy we used was the instructor-provided dataset.

Section C: Machine Learning Model

The initial choice of our machine learning model was PyTorch MLP. However, the output was poor. We decided to switch to a CNN model, which performed much better and was able to analyze the datasets with much larger accuracy.